SOFTWARE ENGINEERING · CHAPTER 09

Activity Diagrams

Activity diagrams model workflows, business processes, and algorithms by showing actions and the flow of control or data.

UML 2.5.1 | Mermaid flowcharts | JDK 25 LTS | Spring Boot 4.1

AGENDA

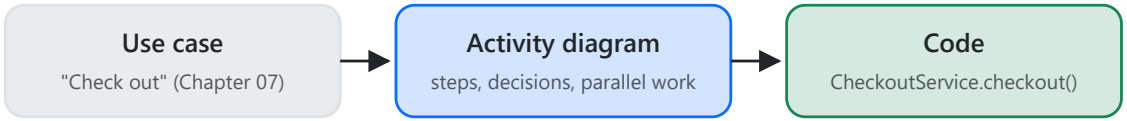
What we will cover

1. Initial and final nodes
2. Actions, activities, and transitions
3. Decisions, merges, forks, and joins
4. Guards and alternative paths
5. Swimlanes and responsibility allocation
6. Object nodes and data flows
7. Iteration and concurrent activities
8. Exceptions and interruption regions
9. Modeling business and software workflows
10. Keeping diagrams readable and consistent
11. Chapter Quiz
12. Lab-09

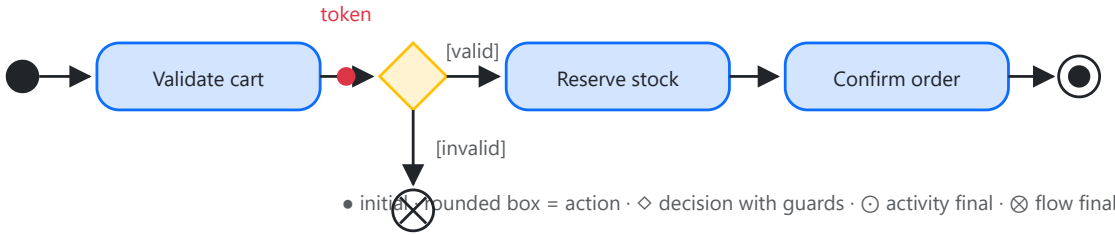
Click any item to jump directly to that topic.

Activity diagrams: behaviour as a flow of tokens

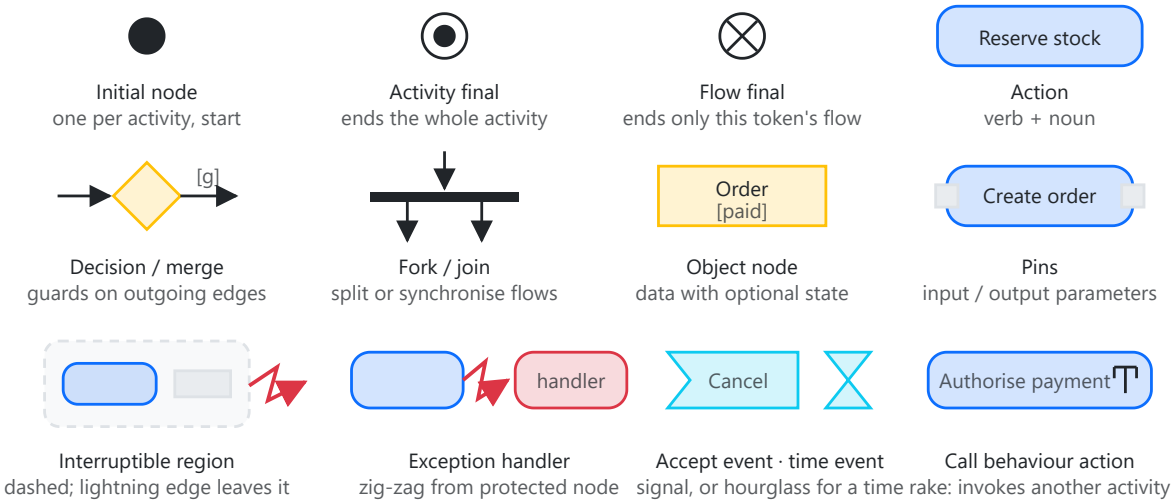
- A **use case** says *what* the system does; an **activity diagram** says *how*, step by step: actions, decisions, parallel work, who does what.
- Semantics: **tokens** travel along edges. An action starts when a token arrives on every incoming edge and emits a token on every outgoing edge when done.
- Three uses in this course: **business processes** (returns handling), **software workflows** (checkout), **algorithms** (binary search).
- Mermaid has no activity diagram type, but a `flowchart` draws the same shapes. We show the Mermaid text next to every diagram.
- Running example: the online shop from Chapters 02 to 08: checkout, cancellation, nightly restock.



One activity per use case; one method (or process) per activity



Notation: initial and final nodes, and everything around them



- **Initial node ●**: where the first token is created. Exactly one per activity in this course (UML allows more, do not use them).
- **Activity final ⊙**: the first token that reaches it terminates *all* flows of the activity. Several activity finals are allowed (success, rejected).
- **Flow final ⊗**: destroys only the arriving token; other branches keep running. Use it to end one branch of a fork or one iteration.
- In code: initial = method entry or process start; activity final = `return`; flow final = the end of one thread's work.

> Mermaid mapping used in this deck: `f-circ` for ●, `fr-circ` for ⊙, `cross-circ` for ⊗, `fork` for the bars, `hourglass` for time events.

TOPIC 1

The simplest activity: initial node, actions, activity final



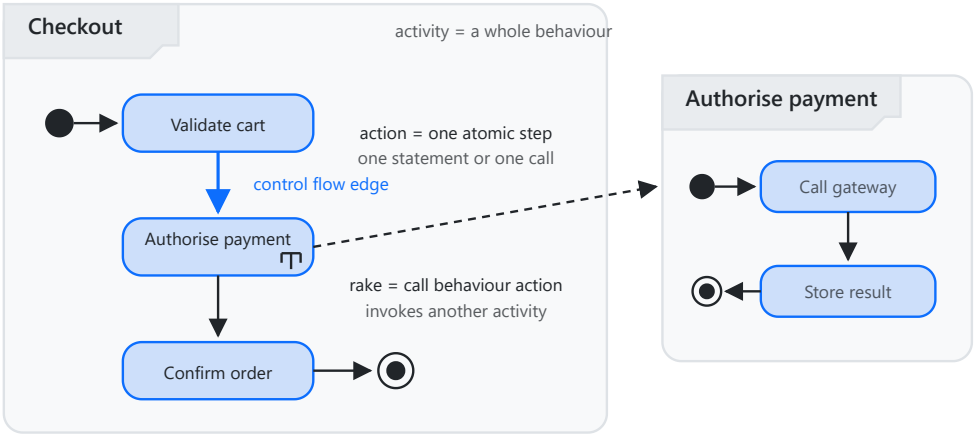
Happy-path checkout: one token enters at ●, passes each action in order, and terminates at ⊙. No branching, no concurrency yet.

```java
@Service
public class CheckoutService {
    // initial node = method entry; activity final = return
    public Confirmation checkout(Cart cart) {
        validator.validate(cart);                  // Validate cart
        Reservation r = stock.reserve(cart.lines()); // Reserve stock
        Payment p = payments.authorise(cart.total()); // Authorise
        return orders.confirm(cart, r, p);         // Confirm order
    }
}
```

```
flowchart LR
  s@{ shape: f-circ, label: " " }
  a1(["Validate cart"])
  a2(["Reserve stock"])
  a3(["Authorise payment"])
  a4(["Confirm order"])
  e@{ shape: fr-circ, label: " " }
  s --> a1 --> a2 --> a3 --> a4 --> e
```

- `(["..."])` draws the rounded action box; the id (`a1`) is not shown.
- `LR` for short linear flows, `TD` once decisions appear (reads like code).
- Every action name is **verb + noun**: "Reserve stock", never "Stock" or "Reservation".
- Each statement in the method corresponds to one action; the order of statements is the order of control flow edges.

Action vs activity, and the edges between them

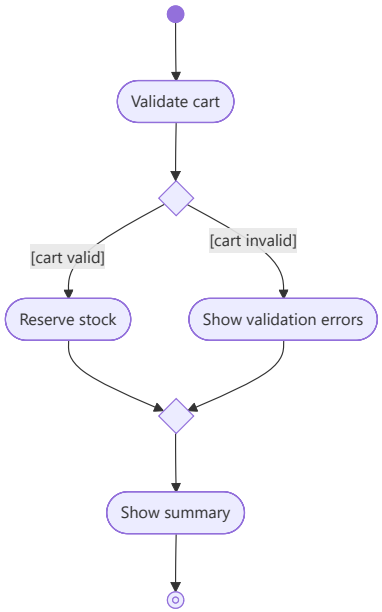


- **Action**: an atomic step that cannot be decomposed further at this level (rounded rectangle).
- **Activity**: the whole behaviour, drawn as a frame; it is made of actions and edges. A method, a process, an algorithm.
- **Call behaviour action** (rake symbol): an action that runs another activity, so diagrams can be nested instead of huge.
- **Transition** is UML 1 vocabulary; UML 2 calls it a **control flow** edge (or object flow when it carries data). It means "when this finishes, that starts".

```
// activity "Checkout" = the method; each action = one call
public Confirmation checkout(Cart cart) {
    validateCart(cart);                  // action
    Payment p = authorisePayment(cart);  // call behaviour action
    return confirmOrder(cart, p);        // action
}

// the called behaviour is its own activity (own diagram)
private Payment authorisePayment(Cart cart) {
    GatewayResult r = gateway.charge(cart.total(), cart.card());
    return payments.save(Payment.from(r));
}
```

Decision and merge: choosing one path, then coming back together

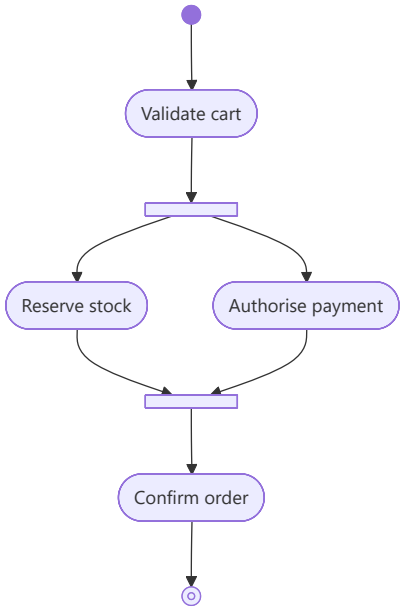



```
flowchart TD
  s@{ shape: f-circ, label: " " }
  v(["Validate cart"])
  d{" "}
  ok(["Reserve stock"])
  ko(["Show validation errors"])
  m{" "}
  n(["Show summary"])
  e@{ shape: fr-circ, label: " " }
  s --> v --> d
  d -->|"[cart valid]"| ok
  d -->|"[cart invalid]"| ko
  ok --> m
  ko --> m
  m --> n --> e
```

```
public Summary checkout(Cart cart) {
    ValidationResult v = validator.validate(cart);
    // decision node: exactly one guard is true
    if (v.isValid()) {
        stock.reserve(cart.lines());
    } else {
        view.showErrors(v.errors());
    }
    // merge node: both paths continue here
    return view.summary(cart);
}
```

- **Decision** ◇: one token in, one token out, chosen by the guards.
- **Merge** ◇: any token in, passed straight on; no waiting, no synchronisation.
- Same diamond symbol; count the edges to tell them apart.

Fork and join: reserve stock *and* authorise payment at the same time



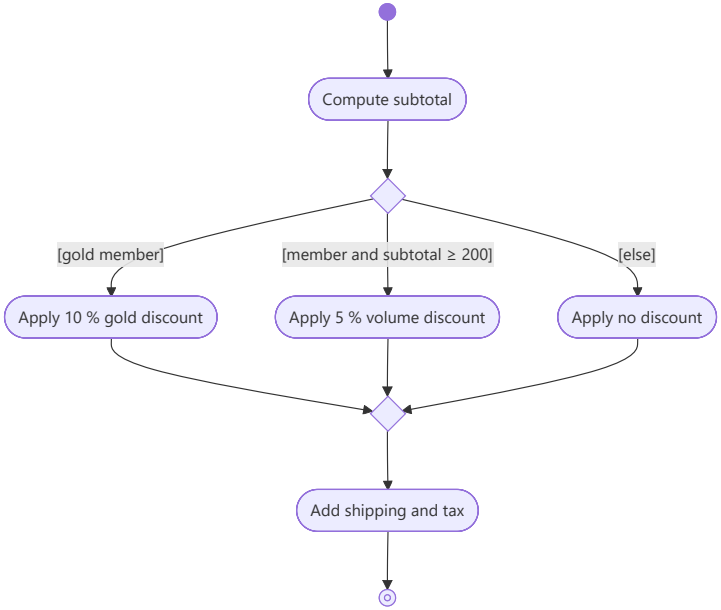

```
f@{ shape: fork, label: " " }
j@{ shape: fork, label: " " }
s --> v --> f
f --> r --> j
f --> p --> j
j --> c --> e
```

- **Fork**: one token in, one token out on *every* outgoing edge: the branches run concurrently.
- **Join**: waits until a token has arrived on *every* incoming edge, then emits one token. Fork and join are the same bar; count the edges.
- Pitfall: an action with two incoming edges is an *implicit join*. If only one branch ever delivers a token, the action never runs. Use a merge when the paths are alternatives.

```
// fork/join with CompletableFuture (any JDK 8+)
var reserve = CompletableFuture.supplyAsync(
        () -> stock.reserve(cart.lines()), pool);
var authorise = CompletableFuture.supplyAsync(
        () -> payments.authorise(cart.total()), pool);
CompletableFuture.allOf(reserve, authorise).join();     // join node
return orders.confirm(cart, reserve.join(), authorise.join());
```

```
// fork/join with structured concurrency (JDK 25, preview, JEP 505)
// compile and run with --enable-preview
public Confirmation checkout(Cart cart) throws InterruptedException {
    try (var scope = StructuredTaskScope.open()) {
        var r = scope.fork(() -> stock.reserve(cart.lines()));
        var p = scope.fork(() -> payments.authorise(cart.total()));
        // join node: returns when both succeeded,
        // throws FailedException as soon as one fails
        scope.join();
        return orders.confirm(cart, r.get(), p.get());
    }
}
```

Guards: boolean conditions that must be complete and exclusive




```
s --> a --> d
d -->|"[gold member]"| g
d -->|"[member and subtotal ≥ 200]"| v
d -->|"[else]"| n
g --> m
v --> m
n --> m
m --> t --> e
```

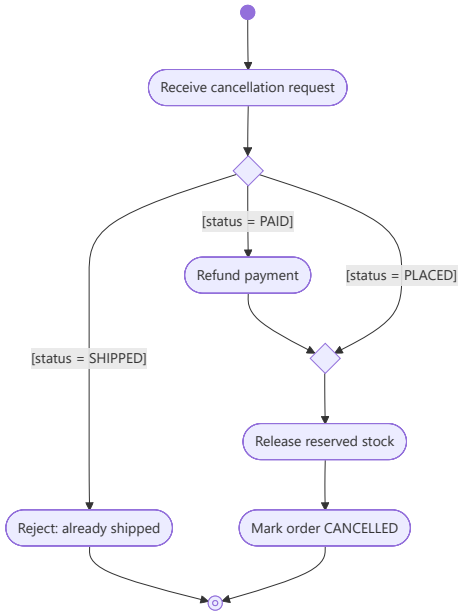
```java
sealed interface Customer permits Guest, Member, GoldMember {}
record Guest() implements Customer {}
record Member(String id) implements Customer {}
record GoldMember(String id) implements Customer {}

static final BigDecimal LIMIT = new BigDecimal("200");

BigDecimal discount(Customer c, BigDecimal subtotal) {
    // guards -> cases: complete (default) and exclusive
    // (the first matching case wins, so order them like the diagram)
    return switch (c) {
        case GoldMember g -> subtotal.multiply(new BigDecimal("0.10"));
        case Member m when subtotal.compareTo(LIMIT) >= 0
                -> subtotal.multiply(new BigDecimal("0.05"));
        default -> BigDecimal.ZERO;
    };
}
```

Rule	Meaning	If violated
Complete	for every input some guard is true	token gets stuck at ◇
Exclusive	at most one guard is true	UML picks any: nondeterminism
`[else]`	true when no other guard is	use it exactly once per ◇

Alternative paths: cancelling an order depends on its state



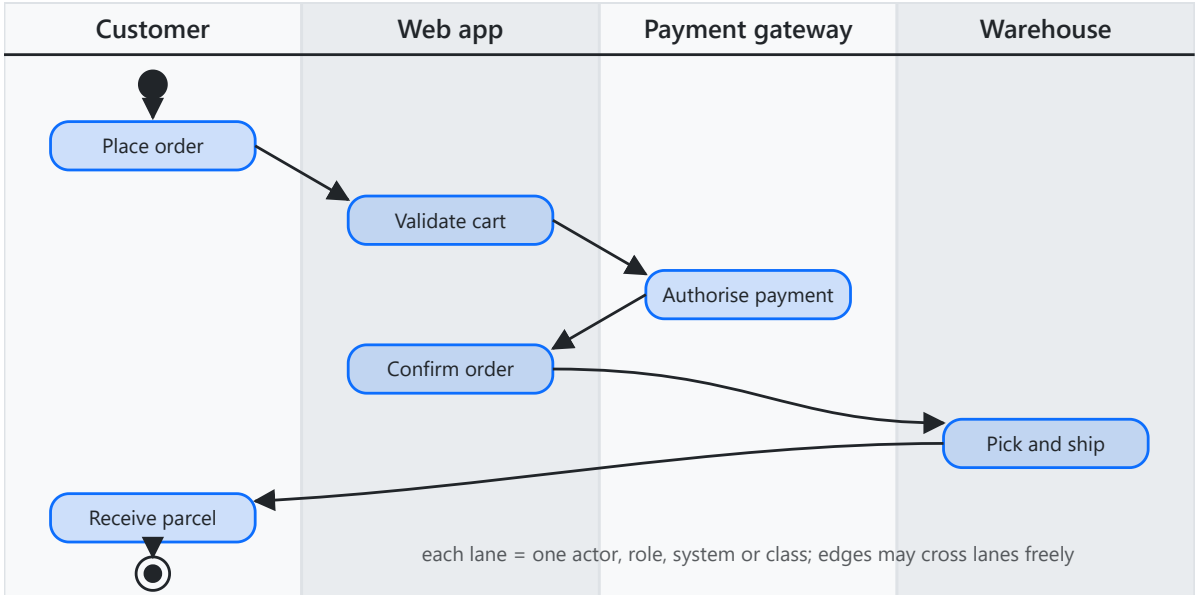

```
  s --> a --> d
  d -->|"[status = PAID]"| p --> m
  d -->|"[status = PLACED]"| m
  d -->|"[status = SHIPPED]"| x --> e
  m --> r --> c --> e
```

```
enum Status { PLACED, PAID, SHIPPED, CANCELLED }

public void cancel(Order order) {
    // decision guards = switch cases; the switch must be exhaustive
    switch (order.status()) {
        case CANCELLED -> { return; }            // nothing to do
        case PLACED    -> { }                    // straight to merge
        case PAID      -> payments.refund(order.paymentId());
        case SHIPPED   ->
                throw new CannotCancelException(order.id());
    }
    // merge node: PLACED and PAID continue here
    stock.release(order.reservationId());        // Release stock
    order.markCancelled();                       // Mark CANCELLED
}
```

⚠ Two edges into *Release reserved stock* would be an implicit **join** and the action would wait forever for the second token. The merge ◇ before it is not optional.

Swimlanes (activity partitions): who performs each action

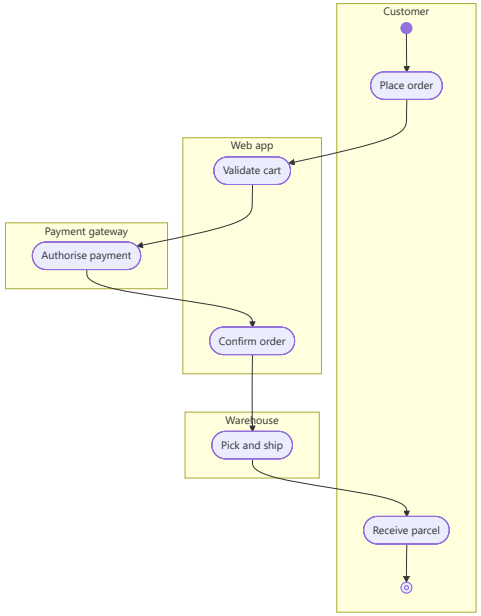


- A **partition** (swimlane) groups the actions performed by one responsible party: a role, a department, a subsystem, a class.
- Lanes do not change the semantics; they show **allocation of responsibility** and hand-offs (edges crossing lanes are interfaces).
- Keep ≤ 5 lanes; more than that, split the diagram or nest partitions.
- In code a lane becomes a class, a service, or an external system behind an interface.

```
// Lane "Web app": orchestrates and owns the decisions
@Service
public class CheckoutService {
    // lanes Payment gateway and Warehouse = interfaces
    private final PaymentGateway gateway;
    private final WarehouseClient warehouse;

    public Confirmation checkout(Cart cart) {
        validate(cart);                    // Web app
        Payment p = gateway.authorise(cart.total());
        Order o = confirm(cart, p);        // Web app
        warehouse.ship(o);                 // Warehouse
        return Confirmation.of(o);         // back to Customer
    }
}
```

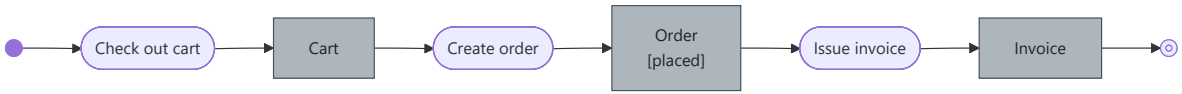
Swimlanes in Mermaid: one `subgraph` per partition




```
flowchart TB
  subgraph C[Customer]
    s@{ shape: f-circ, label: " " }
    a1(["Place order"])
    a6(["Receive parcel"])
    e@{ shape: fr-circ, label: " " }
  end
  subgraph W[Web app]
    a2(["Validate cart"])
    a4(["Confirm order"])
  end
  subgraph P[Payment gateway]
    a3(["Authorise payment"])
  end
  subgraph H[Warehouse]
    a5(["Pick and ship"])
  end
  s --> a1 --> a2 --> a3 --> a4
  a4 --> a5 --> a6 --> e
```

Mermaid places subgraphs where the layout engine likes; draw.io, PlantUML (`|Customer|`) and Visual Paradigm draw real columns. Submit the text either way: it is diff-able and reviewable.

Object nodes: the data that flows from action to action



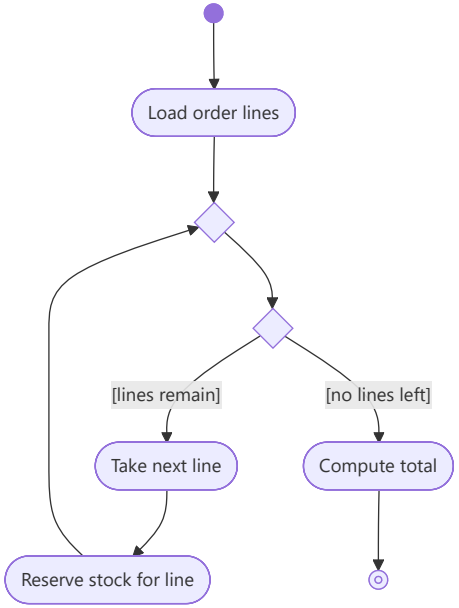

```
// object nodes = the types that flow between actions
public record Cart(List<Line> lines, CustomerId customer) {}
public record Order(OrderId id, Status status, Money total) {}
public record Invoice(InvoiceId id, OrderId order, Money amount) {}

// each action consumes one object and produces the next one
Cart cart = carts.checkOut(customerId);      // -> Cart
Order order = orders.create(cart);           // -> Order [placed]
Invoice invoice = billing.issue(order);      // -> Invoice
```

```
flowchart LR
  s@{ shape: f-circ, label: " " }
  a1(["Check out cart"])
  o1["Cart"]
  a2(["Create order"])
  o2["Order<br/>[placed]"]
  a3(["Issue invoice"])
  o3["Invoice"]
  e@{ shape: fr-circ, label: " " }
  s --> a1 --> o1 --> a2 --> o2
  o2 --> a3 --> o3 --> e
  class o1,o2,o3 obj
```

- **Object node**: a sharp-cornered rectangle named after a *class from the class diagram* (Chapter 08), optionally with a state in brackets: `Order [placed]`.
- **Object flow**: an edge that carries an object token instead of a control token; the action downstream cannot start until the object arrives.
- **Pins**: the small squares on an action are the compact form: the action's parameters and results.
- **Data store** node («datastore» Orders) keeps a copy: a repository or table.

Iteration: a loop is a merge, a decision and an edge going back

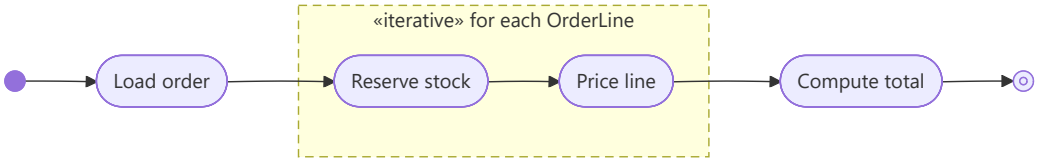



```
flowchart TD
  s@{ shape: f-circ, label: " " }
  a(["Load order lines"])
  m{" "}
  d{" "}
  n(["Take next line"])
  r(["Reserve stock for line"])
  t(["Compute total"])
  e@{ shape: fr-circ, label: " " }
  s --> a --> m --> d
  d -->|"[lines remain]"| n --> r --> m
  d -->|"[no lines left]"| t --> e
```

```
// merge + decision + back edge  ==  a while / for loop
Money total = Money.ZERO;
for (OrderLine line : order.lines()) {     // [lines remain]
    stock.reserve(line.sku(), line.qty());  // Reserve stock for line
    total = total.plus(line.price());
}
// [no lines left]
return total;                               // Compute total
```

- The merge *before* the decision receives both the first token and the back edge: it is what makes the loop legal.
- The loop condition lives on the decision's guards; `[lines remain]` / `[no lines left]` are complete and exclusive.

Expansion regions: "for each element" without drawing the loop




```
// «iterative» expansion region  ==  a sequential stream pipeline
List<PricedLine> priced = order.lines().stream()
        .map(stock::reserve)                    // Reserve stock
        .map(pricing::price)                    // Price line
        .toList();

// «parallel» expansion region  ==  parallel stream (or an executor)
List<PricedLine> pricedFast = order.lines().parallelStream()
        .map(pricing::price)
        .toList();

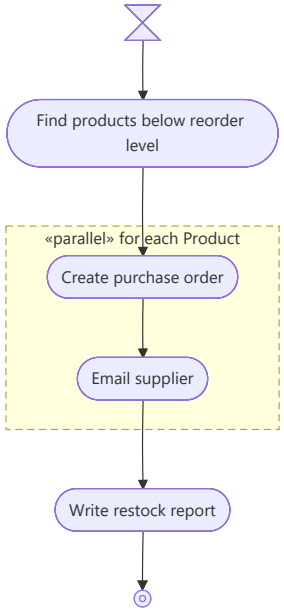
Money total = priced.stream()                   // Compute total
        .map(PricedLine::amount)
        .reduce(Money.ZERO, Money::plus);
```

```
flowchart LR
  s@{ shape: f-circ, label: " " }
  a(["Load order"])
  subgraph X["«iterative» for each OrderLine"]
    r(["Reserve stock"]) --> p(["Price line"])
  end
  t(["Compute total"])
  e@{ shape: fr-circ, label: " " }
  s --> a --> r
  p --> t --> e
  style X stroke-dasharray: 6 4
```

Mode	Semantics	Java
«iterative»	elements one after another	`stream()`, `for`
«parallel»	elements concurrently	`parallelStream()`, executor
«stream»	elements as they arrive	reactive / message consumer

The dashed box takes a collection in and produces a collection out; the actions inside run once per element.

Concurrent activities: the nightly restock job



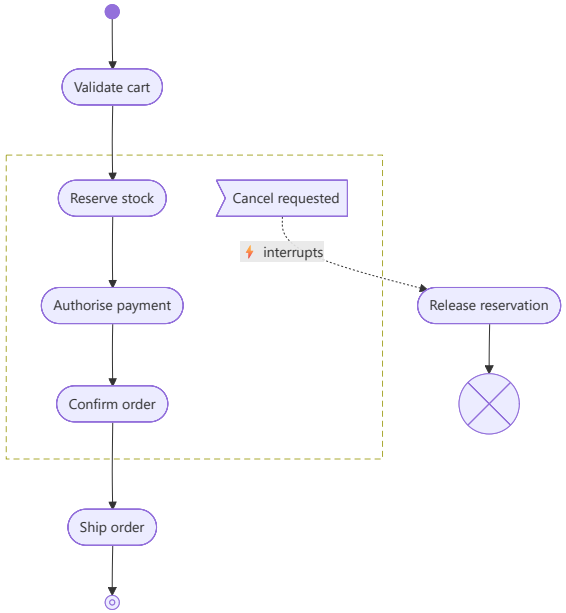

```
flowchart TD
  t@{ shape: hourglass, label: "02:00 every night" }
  a(["Find products below reorder level"])
  subgraph X["«parallel» for each Product"]
    b(["Create purchase order"]) --> c(["Email supplier"])
  end
  d(["Write restock report"])
  e@{ shape: fr-circ, label: " " }
  t --> a --> b
  c --> d --> e
```

```java
@Component
public class NightlyRestockJob {
    private final ProductRepository products;
    private final PurchasingService purchasing;
    private final ReportService reports;
    // constructor omitted

    // accept time event (hourglass): needs @EnableScheduling
    @Scheduled(cron = "0 0 2 * * *", zone = "Asia/Phnom_Penh")
    public void restock() {
        List<Product> low = products.findBelowReorderLevel();
        // «parallel» expansion region
        low.parallelStream().forEach(p -> {
            PurchaseOrder po = purchasing.createOrder(p);
            purchasing.emailSupplier(po);
        });
        reports.writeRestockReport(low);      // after all finished
    }
}
```

An activity does not need an initial node when it starts from an **accept time event**: the hourglass is the token source. The region's end acts as a join: the report is written only after every product was processed.

Interruptible activity regions: "unless the customer cancels"



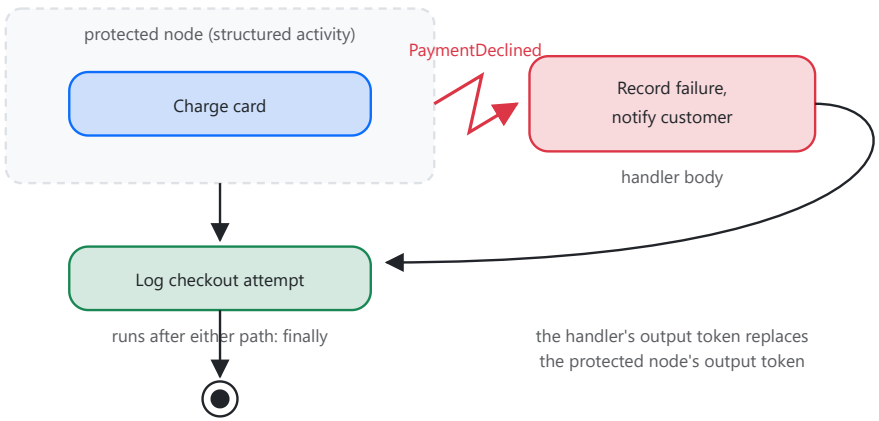

```
  subgraph IR[" "]
    r(["Reserve stock"]) --> p(["Authorise payment"])
    p --> c(["Confirm order"])
    ev>"Cancel requested"]
  end
  x(["Release reservation"])
  ff@{ shape: cross-circ, label: " " }
  s --> v --> r
  c --> sh --> e
  ev -. " ϟ interrupts" .-> x --> ff
  style IR stroke-dasharray: 6 4,fill:none
```

- The dashed region contains an **accept event action** (*Cancel requested*). When the event fires, every token inside the region is destroyed and one token leaves along the lightning-bolt **interrupting edge**.
- Outside the region (*Ship order*) the cancellation has no effect any more.
- The compensating path ends in a flow final ⊗: the rest of the system continues.

```
// region = a cancellable task; accept event = Future.cancel(true)
Future<Order> placing = pool.submit(() -> placeOrder(cart));
cancelButton.onClick(() -> placing.cancel(true));  // interrupt

Order placeOrder(Cart cart) throws InterruptedException {
    Reservation r = stock.reserve(cart.lines());
    try {
        Payment p = payments.authorise(cart.total()); // blocks
        return orders.confirm(cart, r, p);
    } catch (InterruptedException e) {
        stock.release(r);               // Release reservation
        throw e;                        // token leaves via ⊗
    }
}
```

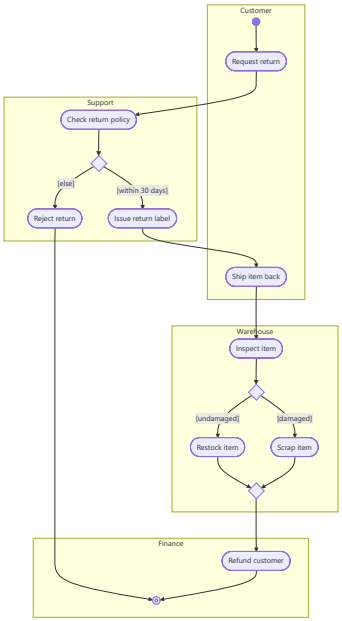
Exception handlers: the zig-zag edge is a `catch` block




```java
public Confirmation checkout(Cart cart) {
    Reservation r = stock.reserve(cart.lines());
    try {                                   // protected node
        Payment p = payments.charge(cart.total(), cart.card());
        return orders.confirm(cart, r, p);
    } catch (PaymentDeclinedException e) { // exception handler
        stock.release(r);                   // Record failure
        notifier.paymentDeclined(cart.customer(), e.reason());
        return Confirmation.declined(e.reason());
    } finally {                             // Log checkout attempt
        audit.log("checkout", cart.id());
    }
}
```

Diagram	Java
Protected node (dashed group)	`try { ... }`
Exception handler + type label	`catch (PaymentDeclinedException e)`
Action after both paths merge	`finally { ... }`
Interruptible region + accept event	`Future.cancel(true)`, `InterruptedException`
Unhandled exception leaving the activity	exception propagates to the caller

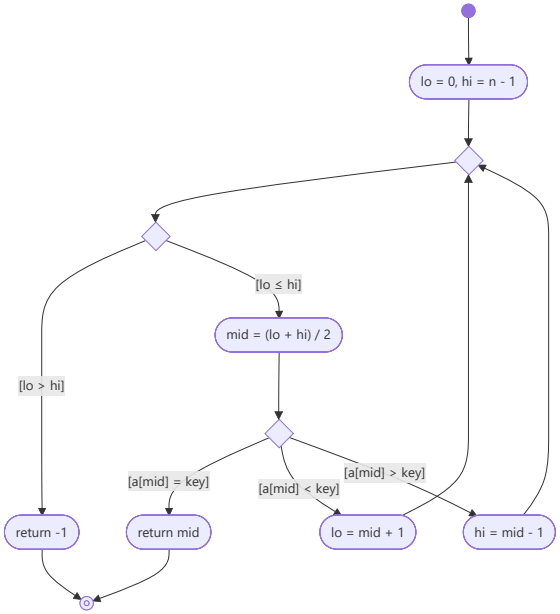
A business process: handling a returned parcel




```
flowchart TB
  subgraph CU[Customer]
    s@{ shape: f-circ, label: " " }
    a1(["Request return"])
    a5(["Ship item back"])
  end
  subgraph SU[Support]
    a2(["Check return policy"])
    d1{" "}
    a3(["Issue return label"])
    a9(["Reject return"])
  end
  subgraph WH[Warehouse]
    a6(["Inspect item"]) ... m{" "}
  end
  subgraph FI[Finance]
    a10(["Refund customer"])
    e@{ shape: fr-circ, label: " " }
  end
  s --> a1 --> a2 --> d1
  d1 -->|"[within 30 days]"| a3 --> a5
  a5 --> a6 --> d2
  d1 -->|"[else]"| a9 --> e
  d2 -->|"[undamaged]"| a7 --> m
  d2 -->|"[damaged]"| a8 --> m
  m --> a10 --> e
```

- Business processes: lanes are **roles and departments**, actions are human or system work, time matters ("within 30 days").
- Two activity finals are fine: *rejected* and *refunded* are both legitimate ends.
- Ask the domain expert to walk a token through the diagram; missing guards show up immediately.

A software algorithm: binary search as an activity diagram




```
s --> a --> m --> d
d -->|"[lo #gt; hi]"| nf --> e
d -->|"[lo ≤ hi]"| b --> c
c -->|"[a[mid] = key]"| f --> e
c -->|"[a[mid] #lt; key]"| l --> m
c -->|"[a[mid] #gt; key]"| h --> m
```

```
static int binarySearch(int[] a, int key) {
    int lo = 0, hi = a.length - 1;        // "lo = 0, hi = n - 1"
    while (lo <= hi) {                    // decision [lo ≤ hi]
        int mid = (lo + hi) >>> 1;        // avoids int overflow
        if (a[mid] == key) return mid;    // [a[mid] = key]
        if (a[mid] < key) lo = mid + 1;   // [a[mid] < key]
        else hi = mid - 1;                // [a[mid] > key]
    }
    return -1;                            // [lo > hi]
}
```

- Three guards on the second decision: complete (=, <, > cover every case) and exclusive.
- Mermaid entities: `#lt;` and `#gt;` inside labels, because `<` would be read as HTML.
- Two activity finals (*found*, *not found*) map to two `return` statements.
- Use activity diagrams for algorithms only when the control flow is the hard part; otherwise the code is the better documentation.

From diagram to code: a `@Service` whose structure mirrors the checkout activity

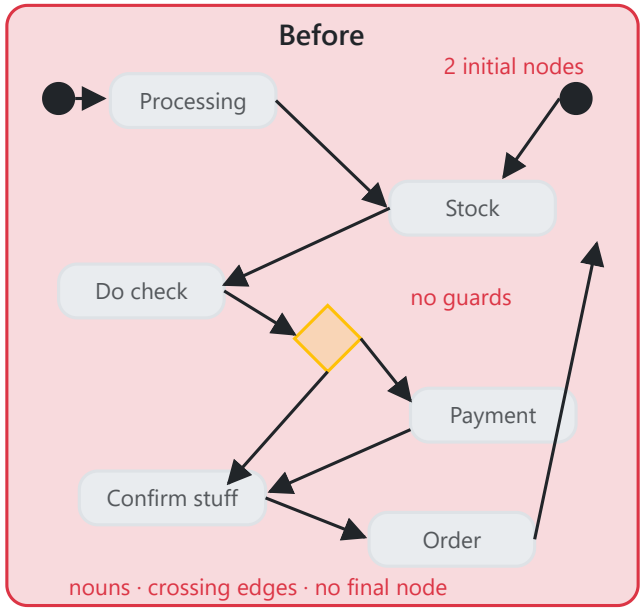
```java
@Service
public class CheckoutService {
    private final CartValidator validator;
    private final StockService stock;
    private final PaymentGateway payments;
    private final OrderRepository orders;
    private final ShippingService shipping;
    // constructor injection omitted for brevity

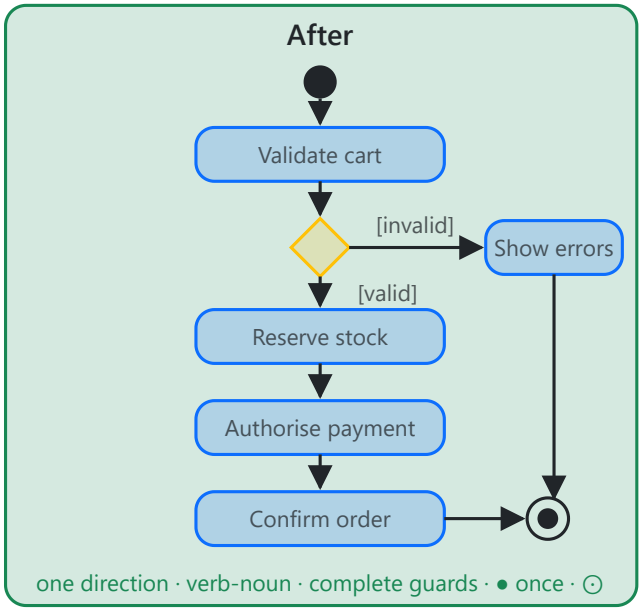
    public Confirmation checkout(Cart cart) throws InterruptedException {
        // 1. Validate cart, then decision [valid] / [invalid]
        ValidationResult v = validator.validate(cart);
        if (!v.isValid()) return Confirmation.rejected(v.errors());
        // 2. fork: Reserve stock  ||  Authorise payment   (JDK 25 preview)
        try (var scope = StructuredTaskScope.open()) {
            var r = scope.fork(() -> stock.reserve(cart.lines()));
            var p = scope.fork(() -> payments.authorise(cart.total()));
            scope.join();                                  // join
            // 3. Confirm order
            Order order = orders.save(
                    Order.confirmed(cart, r.get(), p.get()));
            // 4. Ship order
            shipping.requestShipment(order);
            return Confirmation.of(order);                 // activity final
        } catch (StructuredTaskScope.FailedException e) {
            // exception handler: one branch failed, undo the other
            stock.releaseAll(cart.lines());
            throw new CheckoutFailedException(e.getCause());
        }
    }
}
```

Activity diagram element	Typical code construct
Initial node ●	method entry, process start
Activity final ⊙	`return` / end of process
Flow final ⊗	end of one task or one iteration
Action	statement, method call
Decision + guards	`if / else`, `switch`
Merge	end of the `if / else` block
Fork / join	`CompletableFuture.allOf`, `StructuredTaskScope`
Loop (merge + decision + back edge)	`while`, `for`
Expansion region	`stream().map()`, `parallelStream()`
Object node / pin	parameter, return value, record
Swimlane	class, service, external system
Exception handler	`catch` block
Interruptible region	`Future.cancel(true)`
Accept time event	`@Scheduled`
Accept signal event	`@EventListener`, message consumer

Reviewers can read the diagram and the method side by side: same step names, same order, same guards. That is the whole point of drawing it first.

Keeping diagrams readable: the same workflow, drawn twice





READABILITY CHECKLIST

- At most **~15 actions** per diagram; beyond that use call behaviour actions.
- **One initial node**; every path reaches a final node.
- **Verb + noun** action names: "Reserve stock", not "Stock".
- Guards **complete and exclusive**; one `[else]` at most.
- **≤ 5 swimlanes**, main flow top-down or left-right, no crossing edges.
- Every fork has its join; alternatives use merge, not an implicit join.

```
// naming carries over to code
// "Reserve stock" and guard [cart valid]
void reserveStock(Order o) {}
boolean cartValid(Cart c) {}
```

Consistency: one vocabulary across use case, class, activity, sequence and code

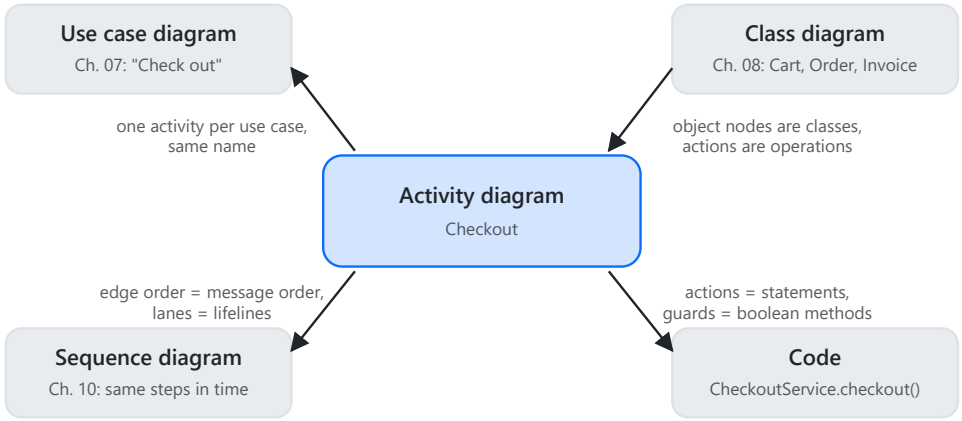


Diagram element	Must agree with	Check
Activity name	use case name	"Check out" in both
Object node `Order [paid]`	class `Order`, enum `Status.PAID`	grep the class diagram
Action "Authorise payment"	operation `PaymentGateway.authorise()`	same verb, same lane
Guard `[subtotal ≥ 200]`	a boolean expression in code	write it as Java before drawing
Swimlane	lifeline in the sequence diagram	same participants

```
// the guard from the diagram, as it appears in code and tests
boolean qualifiesForVolumeDiscount(Customer c, BigDecimal subtotal) {
    return c instanceof Member && subtotal.compareTo(LIMIT) >= 0;
}

@Test void volumeDiscountAppliesAtExactly200() {
    assertTrue(qualifiesForVolumeDiscount(new Member("m1"),
                                          new BigDecimal("200")));
}
```

Keep the Mermaid text in the repository next to the code it describes and review both in the same merge request.

WRAP-UP

Best practices for activity diagrams

1 Start from a use case

Draw one activity per use case (or per service method). Name it identically. The main scenario is the happy path; the extensions become guards and handlers.

2 Write guards as code first

`[status = PAID]` is a boolean expression you will type into an `if` or `switch`. If you cannot write it, you do not understand the decision yet.

3 Fork only what is independent

Reserve stock and authorise payment do not need each other, so they may fork. Confirm order needs both, so it sits after the join.

4 Show the data when it matters

Add object nodes (Cart, Order [placed], Invoice) when the reader must know what is produced; leave them out of a pure control-flow sketch.

5 Model the unhappy paths explicitly

Cancellation, declined payment, invalid rows: interruptible regions and exception handlers are cheaper to draw than to discover in production.

6 Keep the text in version control

Mermaid or PlantUML source lives next to the code, is reviewed in the same merge request and re-rendered by CI. Exported PNGs are for readers, not the source of truth.

Common mistakes and how to fix them

Mistake	Why it hurts	Fix
Two edges entering an action for *alternative* paths	implicit join: the action waits for both tokens and never runs	put a merge ◇ in front of it
Guards `[ok]` / `[not ok]` with no definition	nobody can implement or test them	write the boolean expression: `[stock ≥ qty]`
Fork without a join (or vice versa)	unclear when the activity ends; tokens leak	every fork has a matching join, or branches end in ⊗
Overlapping guards `[x > 10]` and `[x > 100]`	nondeterministic choice	make them exclusive: `[10 < x ≤ 100]`
Nouns as action names ("Payment", "Stock")	reader cannot tell what happens	verb + noun: "Authorise payment"
One giant diagram (40+ actions)	unreadable, never updated	call behaviour actions and sub-activities
Using activity finals where flow finals are meant	⊙ kills the other concurrent branch	end a single branch with ⊗
Drawing a state machine as an activity	states are not actions; events are not edges	use a state diagram for one object's lifecycle

```
// Overlapping guards, as they end up in code (bug):
if (x > 10)  applySmallDiscount();
if (x > 100) applyBigDiscount();     // both run for x = 150

// Exclusive guards, same as a well-drawn decision node:
if (x > 100)      applyBigDiscount();
else if (x > 10)  applySmallDiscount();
else              applyNoDiscount(); // [else] path
```

ⓘ **Activity vs state diagram**: "Validate cart → Reserve stock" is an activity (things happen). "PLACED → PAID → SHIPPED" is the state machine of one `Order`. If your boxes are adjectives or nouns and your edges are events, you are drawing the wrong diagram.

Chapter quiz 10 questions

1. Which symbol is the *activity final* node?

- A filled circle
- A circle with a filled dot inside
- A circle with an X inside
- A thick horizontal bar

2. What is the difference between a flow final ⊗ and an activity final ⊙?

- There is none, they are synonyms
- A flow final ends only the arriving token; an activity final terminates all flows
- A flow final may appear only inside swimlanes
- An activity final is used only in business processes

3. Where is the condition of a decision node written?

- Inside the diamond
- On the outgoing edges as guards in square brackets
- On the incoming edge
- In a note attached to the diamond

4. A merge node …

- waits for a token on every incoming edge
- passes any arriving token straight on without synchronising
- duplicates the token on every outgoing edge
- is drawn as a thick bar

5. An action with two incoming control flow edges, where only one path is ever taken, …

- runs as soon as either token arrives
- is an implicit join and never runs
- runs twice
- is a syntax error in UML

6. Guards on one decision node must be …

- complete and mutually exclusive
- written in natural language only
- at most two
- ordered alphabetically

7. Which Java construct corresponds to a fork followed by a join?

- if / else
- CompletableFuture.allOf or a StructuredTaskScope
- a for loop
- try / finally

8. What does a swimlane (activity partition) express?

- The time at which an action runs
- Who or what is responsible for the actions inside it
- The exception type of a handler
- The loop condition

9. What happens when the accept event inside an interruptible region fires?

- Nothing until the region finishes
- All tokens inside the region are destroyed and the flow continues along the interrupting edge
- The activity restarts from the initial node
- A join is executed

10. Which element maps naturally to a Spring `@Scheduled` method?

- An object node
- An accept time event (hourglass)
- A merge node
- A pin

WRAP-UP

Summary

Tokens flow from one initial node ● to an activity final ⊙; a flow final ⊗ ends only one branch.	Fork and join model concurrency: `CompletableFuture.allOf` or `StructuredTaskScope` (JDK 25 preview).	Loops are merge + decision + back edge; expansion regions are `stream()` / `parallelStream()`.
Actions are atomic steps named verb + noun; an activity is the whole behaviour, nested via call behaviour actions.	Guards must be complete and mutually exclusive; write them as boolean expressions before drawing.	Interruptible regions and exception handlers map to cancellation and `try/catch/finally`; time events to `@Scheduled`.
Decision + guards choose one path (`if`/`switch`); a merge brings alternatives back without waiting.	Swimlanes allocate responsibility (≤ 5 lanes); object nodes show the data (Cart → Order → Invoice).	Keep diagrams ≤ ~15 actions, consistent with use case, class and sequence diagrams, and store the text in git.

Open Lab-09: 5 tasks + 1 challenge

Next chapter: 10 · Sequence Diagrams